

# Release Engineering, Reimagined

On a mission-critical communications program, I was one of three software integrators on the team — carrying the largest share of prototype hardware — and releasing a new software version was a manual, document-heavy process that didn't travel well past a desk. I redesigned it end to end: **templated, portable release notes** that render anywhere, and **a one-click tool** that replaced hours of manual versioning work, alongside a stream of smaller Python, Bash, and Batch scripts that removed everyday friction for the whole team.

## THE PROBLEM

Release notes were written and maintained as Word documents — accurate, but painful to keep that way. Every release meant re-editing a formatted document by hand, and the result only worked well on a machine with Office installed and configured to match. That made it a poor fit for a lab floor: nobody wants to open a laptop and hunt for the right version of Word just to check what changed. It didn't display cleanly on shared lab screens, wasn't easy to pull up on a phone during a walk-through, and every editing pass risked breaking the formatting for the next person.

Creating a new version was its own separate manual process — done by hand, one step at a time, with no tooling behind it.

## WHAT I BUILT

BEFORE

- Release notes hand-edited in Word each cycle
- Needed Office installed to open or edit
- Didn't render well on shared screens or phones
- New versions created and packaged by hand

→

AFTER

- Release notes built from an HTML template, editing only what changed
- Converted to PDF by a Linux script — opens anywhere, no software required
- Displayed cleanly on lab TVs, shared screens, and phones
- New versions created, named, packaged, and encrypted with one command

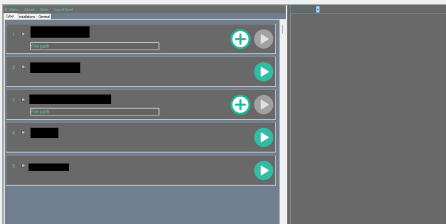
**Portable release notes.** I moved the release-note format from a Word document to a pre-built HTML template — the layout and tables stay fixed, and only the content that actually changes between releases gets edited. I then wrote a Linux script that renders that HTML into a PDF automatically. A PDF opens identically on any device with no editing software required, so the same release note could go up on a lab TV, get shared in a group session on a shared computer, or get pulled up on someone's phone mid-conversation — something the old Word-based version never handled well.

**One-click versioning.** Creating a new release used to be a fully manual sequence. I built a tool that turns it into an interactive checklist: it walks through what's being included in the release, then on confirmation automatically names and numbers the new version in sequence, packages it, produces an encrypted build alongside the standard one, and generates that version's release note from the same HTML template — all from a single command instead of a manual, multi-step process.

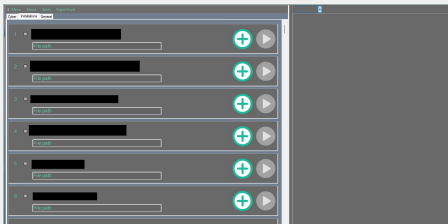
## REMOTE BURNER APP (RBA)

Our team worked day to day on a Windows host machine, but builds actually ran inside a Linux container on that same box. Getting a new build onto it meant SSH-ing in by hand, transferring the file, extracting it, and running the install command — a repetitive, error-prone sequence that every engineer on the team had to do themselves, every time.

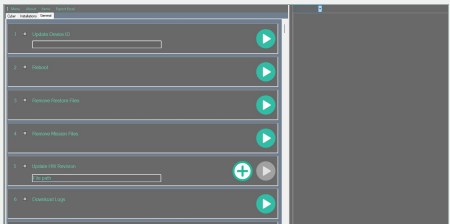
I built RBA, a Windows desktop application that hides all of that behind a simple interface: the user just picks a file and presses a button. Behind the scenes, RBA handles the SSH transfer to the Linux side automatically, runs the remote extraction and installation, and cleans up the leftover files afterward — collapsing a manual, multi-step SSH workflow into a single click. It grew to cover cyber/security operations, product and firmware installation tasks, and general device operations (device ID, reboot, log collection) side by side in one tool, and labs also picked it up to verify connectivity between prototype units over a network link — beyond what I originally built it for.



Cyber tab — security-related device operations, each with one-click execution



Installations tab — product/firmware install tasks, file selected once and pushed automatically



General tab — device ID, reboot, log collection, and other routine device operations

## EVERYDAY SCRIPTING

---

Beyond the two larger tools, I wrote a steady stream of smaller Python, Bash, and Batch scripts aimed at removing day-to-day friction — for myself and for the other integrators on the team. As one of three integrators covering the most prototype hardware on the team, small time sinks repeated across every unit added up fast.

- **Process status at a glance.** Checking whether a given process was actually alive on a prototype used to mean checking it manually, one at a time. I wrote a shell alias that lists every process at once with a clear RUNNING / NOT RUNNING label, color-coded green or red, so system status became a single glance instead of a manual check per process.

## BEYOND THE TOOLING

---

- **Validation.** Every version I produced was run through sanity checks on our prototype hardware before it went anywhere near a lab or customer, working directly with my manager to hold a consistent quality bar release over release.
- **Internal support.** Provided ongoing hands-on support to lab colleagues and engineering teams working with these releases day to day.
- **Customer-facing support.** Traveled internationally to support customers directly on-site — deployment, troubleshooting, and training on our products in the field.

**Skills** — Python · Bash · Linux · HTML/CSS templating · Automated PDF generation · Release & version management · Encryption tooling · Windows desktop application development · SSH automation · Cross-functional collaboration · On-site technical support